

```
type: Assignment
track: "[[General AI Fluency]]"
week: 2
phase: Foundations
status: ● Complete
tags:
date: 14/07/2026
name: Prompting Fundamentals on Real Tasks v2
code: FL-02
```

FL-02 Prompt Iteration Log

Task

Create a reusable prompt that helps analyze a CTF challenge using an evidence-first workflow.

The goal is not to ask AI to solve the whole challenge in one message. The goal is to make AI help with triage: read the available evidence, avoid guessing, and suggest the next useful action.

This task comes from my FL-01-style work around cybersecurity, reverse engineering, binary exploitation, forensics, and CTF challenge solving.

Version 0 — Naive baseline

Technique used

None. This is the weak one-line prompt I would have used before the track.

Prompt

```
Help me solve this CTF challenge.
```

Output excerpt

```
Sure. First, identify the category of the challenge. If it is reverse
engineering, open it in Ghidra or IDA. If it is pwn, run checksec and look for
buffer overflows. If it is forensics, run strings, binwalk, and exiftool.
Search for the flag format with grep.
```

Note

This output is generic. It gives a common CTF checklist, but it does not reason from evidence. It does not ask what files I have, what the category is, what I already tried, or what output I observed.

It could apply to almost any CTF challenge. The biggest failure is that it jumps straight into tools without understanding the artifact.

Version 1 — Role assignment

Technique used

Role assignment

Prompt

```
You are a senior CTF player helping me analyze a challenge. Help me solve this CTF challenge.
```

Output excerpt

```
As a senior CTF player, I would first avoid guessing the solution. Start by identifying the challenge category, provided files, and any visible behavior. For a binary, run file and checksec. For a web challenge, inspect requests and responses. For forensics, identify the file type and metadata.
```

Note

Adding the role made the answer slightly more disciplined. It stopped sounding like a beginner checklist and started warning against guessing too early.

The output improved because it mentioned identifying the category and visible behavior first. But it still gave a broad menu of options instead of asking for specific evidence. It still did not force one next action.

Version 2 — Context and motivation

Technique used

Context and motivation

Prompt

You are a senior CTF player helping me analyze a challenge.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I want an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Motivation: I do not want generic advice. I want help deciding the next useful analysis step without jumping to a full solve.

Output excerpt

Use an evidence-first triage process.

For every step, separate:

- Evidence: what was observed
- Interpretation: what it may mean
- Confidence: low, medium, or high
- Next action: the smallest check that reduces uncertainty

Example: if checksec shows NX enabled and no canary, that suggests stack execution is blocked but control-flow hijack may still be possible. The next step would be to inspect input handling before assuming the exploit path.

Note

This version improved a lot. The output finally matched the way I want to work: evidence first, no unsupported conclusions, and no full solve plan.

The main improvement is that the model introduced a structure for thinking: evidence, interpretation, confidence, next action. The remaining failure is that it still talks generally. It has not committed to a clear answer format.

Version 3 — Output structure

Technique used

Output structure

Prompt

You are a senior CTF player helping me analyze a challenge.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I want an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Motivation: I do not want generic advice. I want help deciding the next useful analysis step without jumping to a full solve.

Use this output structure:

1. Current evidence
2. What the evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is useful
7. What result I should paste back

Output excerpt

1. Current evidence

No artifact, category, command output, or challenge description has been provided yet.

2. What the evidence suggests

There is not enough evidence to classify the challenge or choose a category-specific path.

3. Confidence level

High confidence that more basic information is needed.

4. What is still unknown

- Challenge category
- Artifact type
- File architecture
- Protections
- Input behavior
- Flag format

5. One next action

Run:

```
file ./challenge
```

6. Why this action is useful

It gives the first hard fact about the artifact and prevents choosing the wrong workflow.

7. What result I should paste back

Paste the full output of the file command.

Note

The output became easier to use because it separated facts, guesses, unknowns, and the next action. It also gave only one next step, which is useful during CTF work.

The weak point is that the format became a bit heavy. For a small challenge, this structure might slow down the interaction. Still, it is better than receiving a generic tool dump.

Version 4 — Step decomposition

Technique used

Step decomposition

Prompt

You are a senior CTF player helping me analyze a challenge.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I want an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Motivation: I do not want generic advice. I want help deciding the next useful analysis step without jumping to a full solve.

Use this output structure:

1. Current evidence
2. What the evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is useful
7. What result I should paste back

Step decomposition rules:

- Do not solve the whole challenge at once.
- Work one step at a time.
- Choose the next step based only on the current evidence.
- After suggesting one action, stop.
- Wait for the result before deciding the next action.

Output excerpt

1. Current evidence

No challenge evidence has been provided.

2. What the evidence suggests

Nothing can be concluded yet. There is no artifact type, command output, service behavior, source code, or category information.

3. Confidence level

High confidence that the next step should collect basic artifact information.

4. What is still unknown

The challenge category, file type, target environment, and likely attack surface.

5. One next action

Run:

```
file ./challenge
```

6. Why this action is useful

It is the smallest step that gives reliable information without assuming the category.

7. What result I should paste back

Paste the full output of:

```
file ./challenge
```

Note

This improved the behavior I care about most. The model stopped giving long roadmaps and committed to one action.

The output is more useful during a real CTF because it acts like an incremental teammate. It waits for evidence instead of trying to impress me with a full plan.

The downside is that it is now strict. If I already have useful evidence, I need to paste it clearly, or the model will restart from basic `file` identification.

Version 5 — Few-shot examples

Technique used

Few-shot examples

Prompt

You are a senior CTF player helping me analyze a challenge.

Context: I usually work on reverse engineering, binary exploitation, forensics, and crypto challenges. I want an evidence-first workflow where every conclusion is tied to something observed from the artifact, command output, decompiled code, logs, memory dump, or network behavior.

Motivation: I do not want generic advice. I want help deciding the next useful analysis step without jumping to a full solve.

Use this output structure:

1. Current evidence
2. What the evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is useful
7. What result I should paste back

Step decomposition rules:

- Do not solve the whole challenge at once.
- Work one step at a time.
- Choose the next step based only on the current evidence.
- After suggesting one action, stop.
- Wait for the result before deciding the next action.

Examples of good behavior:

Example 1:

Input evidence:

"file says the artifact is a 64-bit ELF. checksec shows NX enabled, no canary, PIE disabled."

Good response:

"The evidence suggests a local Linux binary with some exploitability signals, but the bug is not proven yet. The next action is to run the binary with a long cyclic input and observe whether it crashes. Paste the crash output or register state."

Example 2:

Input evidence:

"strings shows many PNG headers inside the file, and binwalk finds embedded zlib streams."

Good response:

"The evidence suggests a file carving or steganography direction. The next action is to extract embedded files with binwalk or carve by offsets, then inspect the extracted outputs. Paste the extraction tree."

Now analyze my current challenge evidence:
[PASTE CHALLENGE EVIDENCE HERE]

Output excerpt

1. Current evidence

The placeholder says "[PASTE CHALLENGE EVIDENCE HERE]", so no real challenge evidence has been provided yet.

2. What the evidence suggests

There is not enough information to identify the category or attack surface.

3. Confidence level

High.

4. What is still unknown

- Whether the challenge is a binary, web app, file, PCAP, crypto script, or remote service
- What tools have already been run
- Whether there is any crash, leak, hidden data, or suspicious behavior

5. One next action

Paste the first available evidence: either the challenge description, the provided file names, or the output of `file` on the main artifact.

6. Why this action is useful

The prompt is designed to reason from evidence. Without evidence, any tool recommendation would be a guess.

7. What result I should paste back

Paste one of these:

- challenge description;
- file list;
- `file ./artifact` output;
- remote host and port behavior;
- source code snippet.

Note

The few-shot examples improved the model's behavior by showing what a good answer should look like. It became more careful and refused to invent details when the evidence placeholder was empty.

This was also the version where I noticed a failure: the model became cautious enough that it did not suggest a direct command immediately. That is not always bad, but it means the quality of the input matters more. If I paste real evidence, the prompt works better.

Final prompt tested on ChatGPT and Claude

Final cleaned prompt

You are a senior CTF player helping me analyze a challenge using an evidence-first workflow.

Goal:

Help me decide the next useful analysis step. Do not try to solve the whole challenge immediately.

Context:

I work on CTF challenges such as reverse engineering, binary exploitation, forensics, crypto, web, and misc. I want every conclusion to be tied to evidence from the artifact, command output, source code, decompiled code, logs, memory dump, network behavior, or challenge description.

Workflow rules:

- Use only the evidence I provide.
- Separate facts from guesses.
- Tie every conclusion to a specific observation.
- Give exactly one next action.
- Do not give a full solve plan.
- Do not list generic tools unless the evidence justifies them.
- If there is not enough evidence, say so clearly.
- Prefer the next action that reduces uncertainty the most.
- After suggesting one action, stop and wait for the result.

Output format:

1. Current evidence
2. What the evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is useful
7. What result I should paste back

Examples:

Example 1:

Input evidence:

"file says the artifact is a 64-bit ELF. checksec shows NX enabled, no canary, PIE disabled."

Good response:

"The evidence suggests a local Linux binary with some exploitability signals, but the bug is not proven yet. The next action is to run the binary with a long cyclic input and observe whether it crashes. Paste the crash output or register state."

Example 2:

Input evidence:

"strings shows many PNG headers inside the file, and binwalk finds embedded zlib streams."

Good response:

"The evidence suggests a file carving or steganography direction. The next action is to extract embedded files with binwalk or carve by offsets, then inspect the extracted outputs. Paste the extraction tree."

Now analyze this challenge evidence:

[PASTE CHALLENGE EVIDENCE HERE]

Cross-model comparison

ChatGPT result

ChatGPT followed the requested structure very closely. It separated current evidence, interpretation, confidence, unknowns, and the next action. It was concise and practical.

The main strength was structure. ChatGPT was good at keeping the response organized and easy to act on. It also respected the "one next action" rule.

The weakness was that it sometimes stayed slightly too template-like. When the evidence was thin, it repeated the format instead of giving a sharper judgment about what was missing.

Claude result

Claude was more cautious and explanatory. It was strong at explaining why a conclusion was or was not supported by evidence.

The main strength was honesty about uncertainty. Claude was less likely to overstate a hypothesis. It also gave more natural reasoning around why a next action mattered.

The weakness was that Claude sometimes wrote more than needed. Even with the "one next action" rule, it tended to include extra explanation and sometimes added small side notes.

Honest comparison

ChatGPT was better for a fast CTF workflow because it produced cleaner structure and a more direct next action.

Claude was better for reasoning quality because it was more careful about uncertainty and unsupported assumptions.

For this task, I would use ChatGPT during active solving when speed matters. I would use Claude when reviewing the reasoning or writing a clean challenge writeup afterward.

Neither model solved the task by itself. The prompt worked best when I gave real command output or artifact evidence.

Final reusable prompt template

You are a senior CTF player helping me analyze a challenge using an evidence-first workflow.

Goal:

Help me decide the next useful analysis step. Do not try to solve the whole challenge immediately.

Challenge information:

- Category: [reverse / pwn / forensics / crypto / web / misc / unknown]
- Artifact or target: [file name, source code, PCAP, memory dump, website, remote service, logs, etc.]
- Flag format: [flag format if known]
- What I already tried: [commands, tools, guesses, failed attempts]
- Current evidence:
[Paste command output, challenge description, file output, checksec output, code snippet, decompiled code, logs, screenshots, or observed behavior.]

Workflow rules:

- Use only the evidence I provide.
- Separate facts from guesses.
- Tie every conclusion to a specific observation.
- Give exactly one next action.
- Do not give a full solve plan.
- Do not list generic tools unless the evidence justifies them.
- If there is not enough evidence, say so clearly.
- Prefer the next action that reduces uncertainty the most.
- After suggesting one action, stop and wait for the result.

Output format:

1. Current evidence
2. What the evidence suggests
3. Confidence level
4. What is still unknown
5. One next action
6. Why this action is useful
7. What result I should paste back

Quality criteria:

- Be concise.
- Be technical.
- Avoid generic CTF checklists.
- Do not assume the vulnerability too early.
- Do not hallucinate challenge details.
- Make uncertainty visible.
- Work incrementally from evidence.

Final reflection

The biggest improvement came from the combination of context, step decomposition, and few-shot examples.

The naive prompt produced generic CTF advice. Role assignment helped slightly, but not enough. Context and motivation made the output match my actual workflow. Output structure made it easier to use. Step decomposition forced the model to stop giving full solve plans. Few-shot examples showed the model what "good" evidence-first reasoning looks like.

The final prompt is reusable because a stranger can replace the category, artifact, evidence, and flag format without needing my personal context.